Your VLA is Secretly kNN

A nearest-neighbour lookup over the demonstrations, with no training, is a probe for vision-language-action models. It measures how much of a benchmark is plain retrieval, tests a VLA's backbone and its action expert separately, and shows what fine-tuning does to each; a neighbour metric computed offline predicts the closed-loop result. LIBERO and LIBERO-plus, 9 LeRobot VLAs.

Denis Grachev · Sereact Research

91.7 / 82.4%
Training-free kNN lookup over the demos, on LIBERO / LIBERO-plus: the smallest drop under perturbation of every policy tested.*

2 of 9
VLAs score below a lookup on their own fine-tuned VLM (4 of 9 under perturbation): backbone and action expert, tested separately.

+5 to +37
points of LIBERO-plus success that fine-tuning adds to a VLM used as a key (7 of 8 VLAs); π_0's fine-tune breaks its key.

$r = 0.78$
an offline neighbour metric tracks backbones' closed-loop success ($\rho = 0.61$); trained as a loss it lifts a lookup 88.7 → 93.0%.

1 A kNN policy

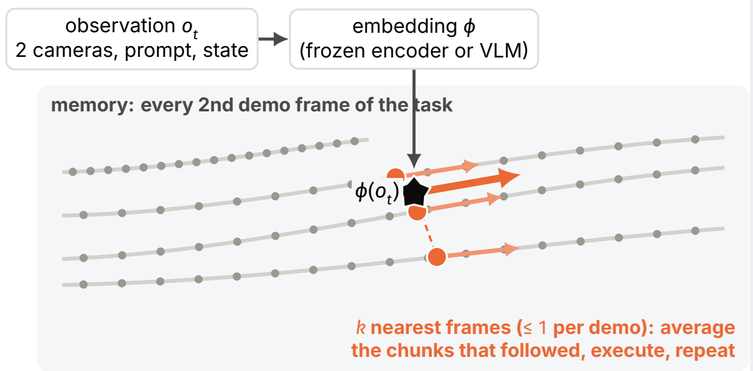
The idea

A policy maps camera images, an instruction and the robot state to a chunk of future actions. A lookup does it with no training: embed the observation with ϕ, find the demonstration frames $(o_i, a_{t_i:t_i+H})$ closest to it, and execute what the demonstrators did next:

$$\pi(o) = \sum_{i \in N_k(\phi(o))} w_i \, a_{t_i:t_i+H}, \qquad w_i \propto e^{-d(o,o_i)/\tau}$$

ϕ can be any encoder: a frozen ImageNet ResNet-18, or the internal states of a VLA's vision-language model, taken before or after its fine-tuning. That makes the lookup a probe for the benchmark (§2) and for the VLA (§3). **It is a LeRobot policy plugin**: fitted from any LeRobot dataset in one command and run with `lerobot-eval`, so **any benchmark LeRobot supports can run the test**.

The kNN policy in LeRobot

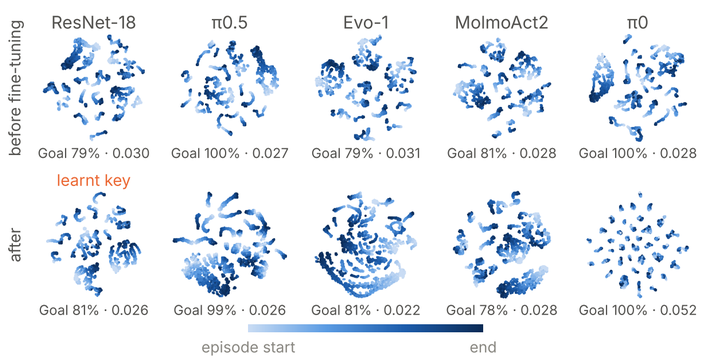


- **Memory**: every 2nd frame of the 1,693 demonstrations (137k keys) with the chunk that followed. "Training" is embedding them: minutes, no gradient step.
- **Key ϕ**: ResNet-18 layer 3 on a 4×4 grid per camera; for a VLA, one layer's hidden states over each camera's image tokens, standardised and pooled to a 2×2 grid.
- **Distance**: image distance per camera, plus state, prompt (CLIP), time-in-episode and history terms, each scaled by the task's median.
- **Retrieve**: the k nearest frames, at most one per demonstration, softmax-weighted.
- **Mode seeking**: neighbours whose chunks disagree with the dominant one are down-weighted instead of averaged into an action no demonstrator took. Every action stays inside its neighbours' convex hull.
- **Temporal ensembling**: overlapping chunk predictions, exponentially weighted (as in ACT).
- **Retry** (option, off): rewind and retrieve again when the scene looks unseen or the arm is stuck; no gain on LIBERO-Long (80.8% → 78.5–81.3%).

Tuning

Optuna searched all of the above: 193 trials of 400 episodes on initial states 40–49 and 0–19 (LIBERO has no held-out split, so the official evaluation includes them, as for every LIBERO result); k and *at most one per demonstration* explain 57% and 25% of the variance. Without the prompt success falls to 64.6%, with raw pixels as the key to 57.2%, with 10% of the demos to 77.4%: **the representation, the prompt and the data do the work.**

What a key encodes



t-SNE of each key on the same 4,150 demo frames; below: same-task neighbours in LIBERO-Goal (one scene, ten instructions) and the median progress gap to the nearest frame of another demo. **Fine-tuning makes VLM keys track the task's phase** (7 of 8; Evo-1 0.031 → 0.022); π_0's keeps the task but loses the phase (0.052): its lookup falls to 17.9%.

2 How hard is the task?

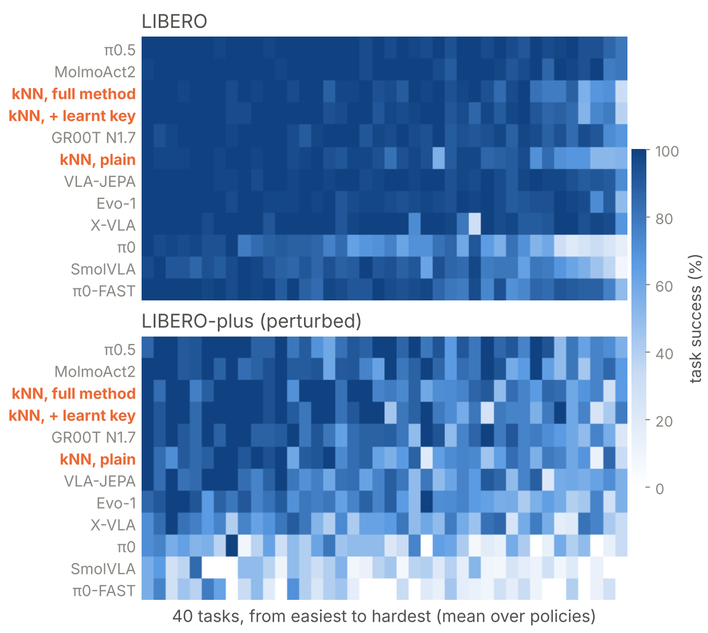
LIBERO: 4 suites × 10 tasks, ~42 demonstrations per task; every policy on the official protocol, 50 episodes per task (2,000).
LIBERO-plus: the same 40 tasks under seven perturbations; a fixed sample of 420 of its 10,030 variants (60 per perturbation), the same for every policy, never trained on.
VLAs: nine public LeRobot Hub LIBERO fine-tunes.

Success on LIBERO and LIBERO-plus

	LIBERO					LIBERO-plus					
	avg	Spat	Obj	Goal	Long	avg	Spat	Obj	Goal	Long	drop
$\pi_{0.5}$	**97.7**	97	100	**98**	**96**	**85.5**	**90**	84	83	**85**	−12.2
MolmoAct2	97.5	**97**	99	98	96	82.6	90	82	76	83	−14.9
kNN, full method	91.7	88	100	98	81	82.4	79	**86**	**87**	78	**−9.3**
kNN, + learnt key	93.0	91	100	97	85	81.7	88	84	81	74	−11.3
GR00T N1.7	94.8	96	98	95	91	80.2	90	74	79	78	−14.6
kNN, plain	88.7	90	95	95	74	75.5	77	82	79	64	−13.2
VLA-JEPA	97.2	95	**100**	97	96	74.0	81	70	77	68	−23.1
Evo-1	95.4	92	99	97	94	71.4	74	73	70	69	−24.0
X-VLA	94.5	95	99	93	92	64.5	71	65	65	57	−30.0
π_0	71.9	70	86	86	45	42.9	54	42	47	29	−29.0
SmolVLA	82.2	79	92	88	70	27.9	30	50	11	20	−54.3
π_0-FAST	90.5	90	94	91	87	25.7	39	34	16	13	−64.8

Ordered by LIBERO-plus success; bold: best per column. The kNN rows are **training-free**, except the learnt key (one linear map per grid cell); *plain*: ImageNet ResNet-18 key, none of the extras. **A training-free lookup is competitive**: 91.7% on LIBERO, rank 7 of 10. **Under perturbation it ranks 3 and degrades least**: 9.3 points, against 12.2 for the most robust VLA and up to 64.8; it is the best of all on two suites.

The same tasks are hard for everyone



VLAs fail where the lookup fails: when a VLA fails a LIBERO episode, the lookup fails the same episode in 20–46% of cases, against 8% overall (2.4–5.5× chance), and per-task success correlates with the lookup's (median $r = 0.74$). The extra episodes the best VLAs solve are a few long-horizon ones. Under perturbation the collapsing VLAs fail almost everywhere.

What breaks under perturbation

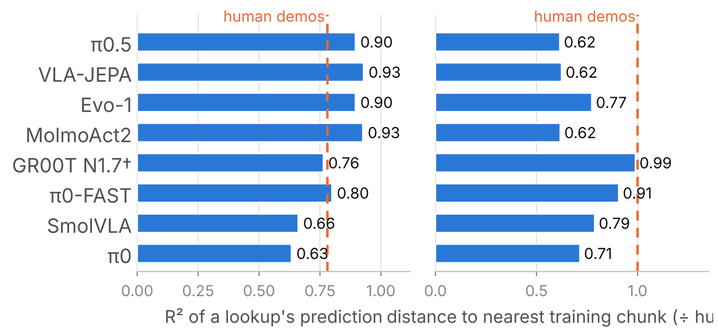
	Camera	Robot	Language	Light	Background	Noise	Layout	all
π0.5	68	78	92	100	93	83	83	**85**
MolmoAct2	73	67	87	95	100	75	82	**83**
kNN, full method	73	58	98	92	97	92	67	**82**
kNN, + learnt key	88	58	90	90	82	88	75	**82**
GR00T N1.7	63	63	87	95	97	85	72	**80**
kNN, plain	68	48	77	92	85	88	70	**75**
VLA-JEPA	55	58	88	92	93	55	77	**74**
Evo-1	53	48	82	88	92	73	63	**71**
X-VLA	7	83	77	93	93	42	57	**65**
π0	67	27	12	43	58	52	42	**43**
SmolVLA	25	13	22	43	45	10	37	**28**
π0-FAST	10	15	33	43	37	0	42	**26**

Weak VLAs key on the wording: reworded instructions over the original scenes drop SmolVLA, π_0-FAST and π_0 to 22, 33 and 12% (original wording: 82, 88, 68%): they memorised 40 instructions. X-VLA fails under a camera shift (7%). The lookup suffers most from new robot initial states and object layouts.

3 Inside the VLAs

VLAs act like a lookup

Clean LIBERO rollouts: a lookup over the demos (ResNet-18 key, $k = 8$) predicting each VLA's chunk at the same state, and the chunk's distance to the nearest training chunk.



A lookup predicts a VLA's actions as well as a human's, or better (R^2 0.90 for $\pi_{0.5}$, 0.78 for human demos), and **its actions are not new**: closer to a training chunk than a new human demo is. On LIBERO-plus the robust VLAs stay lookup-like (R^2 0.85 for $\pi_{0.5}$), the collapsing ones do not (-0.01 for π_0-FAST).
†Spatial suite; X-VLA (absolute poses) left out.

Backbone and action expert, before and after fine-tuning

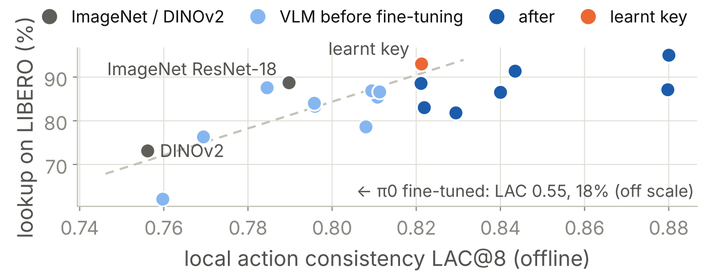
A lookup on a VLA's own VLM scores its **backbone**; the VLA's excess over it is its **action expert**'s (VLM before / after LIBERO fine-tuning, %):

	LIBERO			LIBERO-plus		
	kNN before	**kNN** after	VLA	**kNN** before	**kNN** after	VLA
$\pi_{0.5}$	85.4	81.8	97.7	60.5	65.7	85.5
MolmoAct2	78.6	91.3	97.5	52.6	81.0	82.6
GR00T N1.7*	87.5	87.5	94.8	66.4	66.4	80.2
VLA-JEPA	84.0	83.0	97.2	49.3	65.7	74.0
Evo-1	62.1	87.1	95.4	35.5	**72.6**	71.4
X-VLA	83.3	88.5	94.5	62.4	**78.1**	64.5
π_0	86.9	17.9	71.9	56.7	2.4	42.9
SmolVLA	76.3	**86.5**	82.2	45.0	**69.3**	27.9
π_0-FAST	86.6	**95.0**	90.5	58.3	**74.0**	25.7

Fine-tuning makes a VLM a more robust key (+5 to +37 on LIBERO-plus, 7 of 8 VLAs; clean: 5); π_0's breaks it (86.9 → 17.9%). **Some action experts subtract** (orange): 2 of 9 VLAs fall below a lookup on their own VLM, 4 of 9 under perturbation (up to 48 points). *GR00T froze its VLM.

Scoring a backbone without rollouts

Local action consistency: LAC@8 = 1 − action distance to a key's 8 nearest frames of other demos / to a random frame of the task (held-out queries).



LAC tracks backbones ($r = 0.78$, $\rho = 0.61$; 12 keys) and flags a broken one (π_0 fine-tuned: 0.55). **As a loss it improves a key**: a linear map per ResNet grid cell lifts the lookup 88.7 → **93.0%**.

Conclusions

We do not propose a lookup for robot control. It is a **simple, training-free test** for three questions:

- **How hard is your task?** If a lookup over the demos solves it (LIBERO: 91.7%), success there measures memorisation: VLAs fail on the same tasks, and lose up to 64.8 points off the demos.
- **How good is your backbone, and what did fine-tuning do to it?** A lookup on its VLM before and after fine-tuning (or LAC, without rollouts). Fine-tuning mostly makes it a more robust key (+5 to +37 under perturbation), or breaks it (π_0).
- **How good is your action head?** Its margin over that lookup: 2 of 9 fall below it.

Report a kNN baseline with every result on a dense-demonstration benchmark.

* LIBERO-plus somewhat favours a lookup: its perturbed scenes need the same actions as the clean demonstrations (its own demos replay them), so the right answer is in memory whenever the key ignores the perturbation.

Code: `lerobot_policy_knn`, a LeRobot policy plugin (`uv sync --extra all`); every number and figure is regenerated from the experiment logs by `make`. VLAs: public LeRobot Hub LIBERO checkpoints, none trained here.